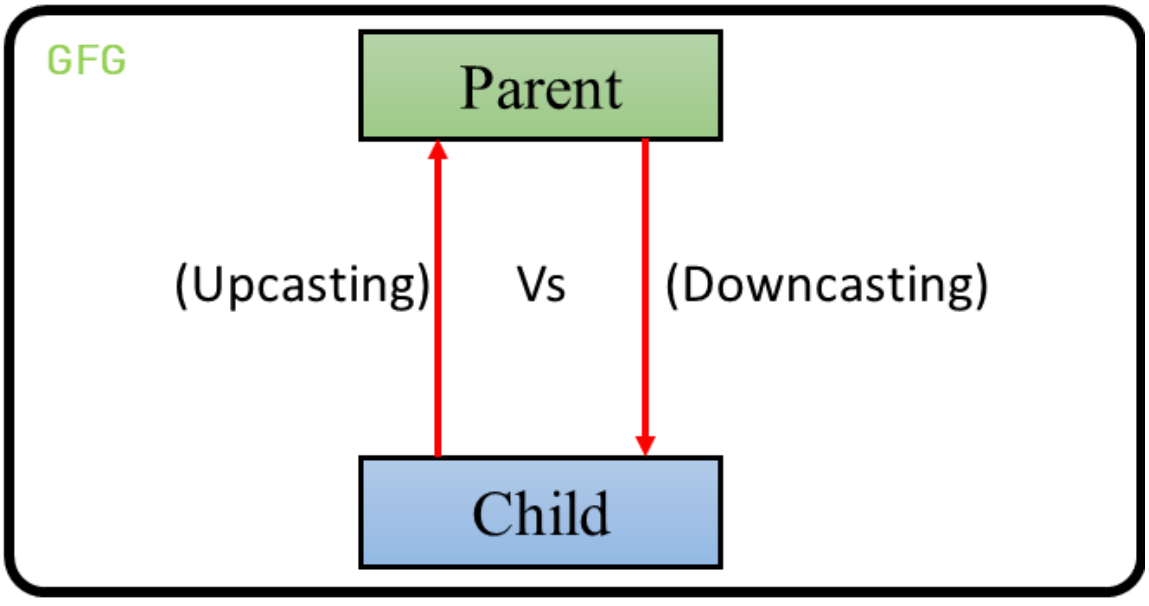
# Upcasting Vs Downcasting in Java

Last Updated : 23 Nov, 2022

Typecasting is one of the most important concepts which basically deals with the conversion of one data type to another datatype implicitly or explicitly. In this article, the concept of typecasting for objects is discussed.
Just like the data types, the objects can also be typecasted. However, in objects, there are only two types of objects, i.e. parent object and child object. Therefore, typecasting of objects basically means that one type of object (i.e.) child or parent to another. There are two types of typecasting. They are:

1. **Upcasting:** Upcasting is the typecasting **of a child object to a parent object**. Upcasting can be done implicitly. Upcasting gives us the flexibility to access the parent class members but it is not possible to access all the child class members using this feature. Instead of all the members, we can access some specified members of the child class. For instance, we can access the overridden methods.
2. **Downcasting:** Similarly, downcasting means the typecasting of a **parent object to a child object**. Downcasting cannot be implicit.

The following image illustrates the concept of upcasting and downcasting:



**Example:** Let there be a parent class. There can be many children of a parent. Let's take one of the children into consideration. The child inherits the properties of the parent. Therefore, there is an "is-a" relationship between the child and parent. Therefore, the child can be implicitly **upcasted** to the parent. However, a parent may or may not inherits the child's properties. However, we can forcefully cast a parent to a child which is known as **downcasting**. After we define this type of casting explicitly, the compiler checks in the background if this type of casting is possible or not. If it's not possible, the compiler throws a ClassCastException. Let's understand the following code to understand the difference:

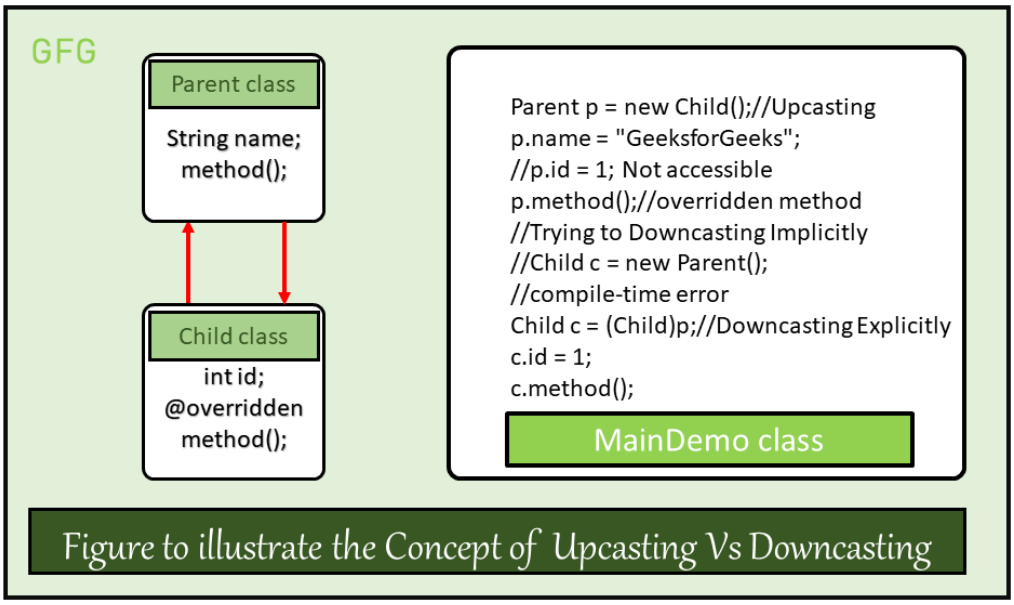
```
// Java program to demonstrate
// Upcasting Vs Downcasting

// Parent class
class Parent {
    String name;

    // A method which prints the
    // signature of the parent class
    void method()
    {
        System.out.println("Method from Parent");
    }
}

// Child class
class Child extends Parent {
    int id;

    // Overriding the parent method
    // to print the signature of the
    // child class
    @Override void method()
    {
        System.out.println("Method from Child");
    }
}

// Demo class to see the difference
// between upcasting and downcasting
public class GFG {
```

```
    // Driver code
    public static void main(String[] args)
    {
        // Upcasting
        Parent p = new Child();
        p.name = "GeeksforGeeks";

        //Printing the parentclass name
        System.out.println(p.name);
        //parent class method is overridden method hence this will be executed
        p.method();

        // Trying to Downcasting Implicitly
        // Child c = new Parent(); - > compile time error

        // Downcasting Explicitly
        Child c = (Child)p;

        c.id = 1;
        System.out.println(c.name);
        System.out.println(c.id);
        c.method();
    }
}
```

### Output

```
GeeksforGeeks
Method from Child
GeeksforGeeks
1
Method from Child
```

### An illustrative figure of the above program:




**From the above example we can observe the following points:**

1. **Syntax of Upcasting:**

```
Parent p = new Child();
```

1. Upcasting will be done internally and due to upcasting the object is allowed to access only parent class members and child class specified members (overridden methods, etc.) but not all members.

```
// This variable is not
// accessible
p.id = 1;
```

1. **Syntax of Downcasting:**

```
Child c = (Child)p;
```

1. Downcasting has to be done externally and due to downcasting a child object can acquire the properties of the parent object.

```
c.name = p.name;
i.e., c.name = "GeeksforGeeks"
```



Next Article

Rules of Downcasting Objects in Java

## Similar Reads

A programmer is always said to write clean codes, where naming has to be appropriate so that for any other programmer it acts as an easy way out to read the code. At a smaller level, this seems meaningles...

5 min read

## Autoboxing and Unboxing in Java

In Java, primitive data types are treated differently so do there comes the introduction of wrapper classes where two components play a role namely Autoboxing and Unboxing. Autoboxing refers to the...

5 min read

## Exception Propagation in Java

Prerequisite : Exceptions in Java, Checked vs Unchecked Exceptions Exception propagation : An exception is first thrown from the top of the stack and if it is not caught, it drops down the call stack to the previou...

3 min read

## Static vs Dynamic Binding in Java

There are certain key points that are needed to be remembered before adhering forward where we will be discussing and implementing static and dynamic bindings in Java later concluding out the differences....

5 min read

## Generalization and Specialization in Java

General class? Loosely speaking, a class which tells the main features but not the specific details. The classes situated at the top of the inheritance hierarchy can be said as General. Specific class? A class...

5 min read

## Static class in Java

Java allows a class to be defined within another class. These are called Nested Classes. Classes can be static which most developers are aware of, henceforth some classes can be made static in Java. Java...

3 min read

## Shadowing of static functions in Java

In Java, if the name of a derived class static function is the same as a base class static function then the base class static function shadows (or conceals) the derived class static function. For example, the...

2 min read

## Covariant Return Types in Java

As the ear hit eardrums "overriding" we quickly get to know that it can be done either virtue of different datatypes or arguments passed to a function what a programmer learned initially while learning...

3 min read

## Cloning in java

Object cloning means to create an exact copy of the original object. If a class needs to support cloning, it must implement java.lang.Cloneable interface and override clone() method from Object class. Syntax of...

1 min read

## Classes and Objects in Java

In Java, classes and objects are basic concepts of Object Oriented Programming (OOPs) that are used to represent real-world concepts and entities. The class represents a group of objects having similar...

11 min read

## Method Hiding in Java

Prerequisite: Overriding in Java If a subclass defines a static method with the same signature as a static method in the superclass, then the method in the subclass hides the one in the superclass. This...

3 min read

## Different ways to create objects in Java

There are several ways by which we can create objects of a class in java as we all know a class provides the blueprint for objects, you create an object from a class. This concept is under-rated and sometimes...

6 min read

## Different name reusing techniques in Java

Overriding An instance method overrides all accessible instance methods with the same signature in superclasses, enabling dynamic dispatch; in other words, the VM chooses which overriding to invoke...

3 min read

## Java - Inner Class vs Sub Class

Inner Class In Java, one can define a new class inside any other class. Such classes are known as Inner class. It is a non-static class, hence, it cannot define any static members in itself. Every instance has...

3 min read

## Passing and Returning Objects in Java

Although Java is strictly passed by value, the precise effect differs between whether a primitive type or a reference type is passed. When we pass a primitive type to a method, it is passed by value. But when w...

6 min read

## Type-Safety and Type-Casting in Java Generics

Generics means parameterized types. The idea is to allow type (Integer, String, ... etc., and user-defined types) to be a parameter to methods, classes, and interfaces. It is possible to create classes that work...

4 min read

**Article Tags :** Java Write From Home

**Practice Tags :** Java